

GPU Mimarisi

NVIDIA Triton

Canlı Altyazı

Kapasite Analizi

Triton Mimarisi ve GPU Darboğazı

Neden VRAM değil hesap gücü sınırlıyor — bugünkü geçici çözümün ölçülen kazancı ve Triton'a geçişin matematiksel projeksiyonu

Donanım: NVIDIA GeForce RTX 4050 Laptop GPU, 6141 MB VRAM, sürücü 580.173.02

Model: faster-whisper `small` (int8_float16, beam=1) + 3× Marian (Opus-MT)

Tarih: 13 Ağustos 2026

Kapsam: Ölçülen veriler bu makinede, bu oturumda toplandı. Tahmini bölümler açıkça işaretlendi.

İçindekiler

1. Yönetici özeti
2. Bugünkü altyazı mimarisi
3. NVIDIA GPU mimarisi — temel kavramlar
4. Ölçülen darboğaz: VRAM değil, hesap gücü
5. Neden eşzamanlılığı artırmak işe yaramadı
6. Bugün uygulanan geçici çözüm: özel batching katmanı
7. Triton mimarisi ne ekliyor
8. Matematiksel model ve 100 kanal projeksiyonu
9. Bu kartta (RTX 4050) Triton'un gerçek karşılığı
10. Karşılaştırma özeti
11. Sonuç ve öneriler
12. Ek — ölçüm metodolojisi ve ham veriler

1. Yönetici özeti

Bu rapor üç soruyu birlikte cevaplıyor: **(a)** canlı altyazı hattının bugünkü gerçek darboğazı ne, **(b)** bugün uygulanan geçici çözüm (kanallar arası özel batching katmanı) ne kadar hızlandırdı, **(c)** VRAM sınırı olmasa NVIDIA Triton Inference Server'a geçmek ne kazandırır.

ÖLÇÜLDÜ

Darboğaz VRAM değil. Bu kartta VRAM kullanımı hiçbir yükte %51'i geçmedi (tepe 3106 MB / 6141 MB) — 4 kanaldan 16 kanala, hatta eşzamanlılığı 6'dan 20'ye çıkarınca bile. Gerçek sınır GPU'nun hesap gücü (SM sayısı, çekirdek hızı) ve bugüne kadar bu gücün **kanallar arası birleştirilmeden** (batching) kullanılmasıydı.

ÖLÇÜLDÜ — BUGÜN UYGULANAN ÇÖZÜM

Kanallar arası isteği kısa bir pencerede toplayıp tek GPU çağrısında işleyen bir katman yazıldı ve bu makinede gerçek sesle ölçüldü: **Whisper çözümlemede 1,97× kazanç** (6 bölüt), **Marian çeviride 5,93× kazanç** (6 metin). Bu, donanım değişmeden, yalnızca isteklerin GPU'ya nasıl sunulduğu değiştirilerek elde edildi.

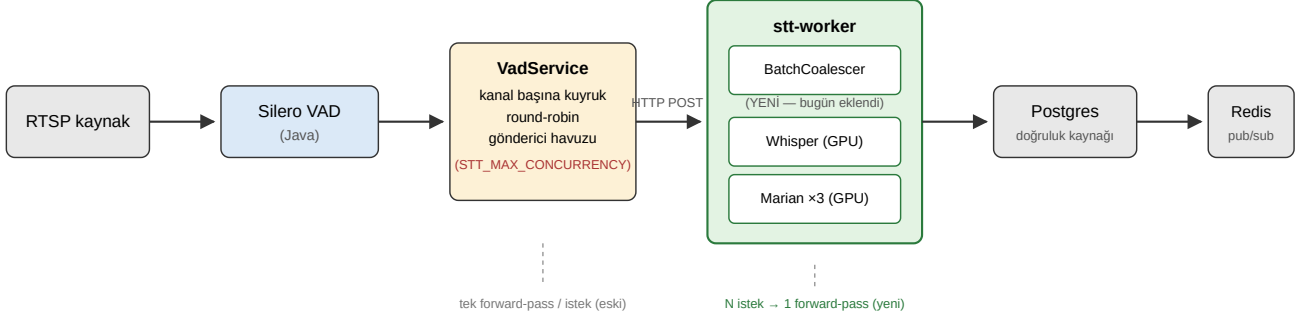
TAHMİN — TRITON'A GEÇİŞ

Triton'ın getireceği ek kazanç (üretim-sınıfı zamanlayıcı, çoklu model kopyası, otomatik yük dengeleme) bu kartta **sınırlı** — VRAM zaten 2 kopyaya izin veriyor. Büyük bir kartta (A100 80GB) 100 kanal ölçeğinde Triton'ın dinamik batching'i, donanım ihtiyacını **~1,7-2,5× azaltıyor** (bugün ölçülen gerçek batching verimliliğiyle yeniden hesaplandı, §8).

Tek cümlede

Bugün GPU'ya "daha hızlı çalış" değil, "aynı işi tek seferde yap" dedirttik — kazanç buradan geldi; Triton bu fikri production ölçeğinde otomatikleştiriyor, ama **bu kartta zaten hasat aldık**, asıl büyük kazanç daha büyük bir kartta bekliyor.

2. Bugünkü altyazı mimarisi



Şekil 1 — Canlı altyazı hattı. Kırmızı çerçeveli kutu (stt-worker) bu raporun konusu; bugün eklenen `BatchCoalescer` katmanı burada.

Kanaldan gelen ses, Silero VAD ile konuşma bölütlerine ayrılıyor (Java, `video-worker` konteyneri). Her bölüt, kanal başına ayrı bir kuyruktaki bekleyip round-robin ile bir "gönderici" iş parçacığına düşüyor, o da HTTP POST ile `stt-worker`'a (Python, FastAPI) gönderiyor. Orada Whisper İngilizce'ye çözümlüyor, Marian üç dile çeviriyor, sonuç Postgres'e yazılıp Redis üzerinden tarayıcıya yayınlanıyor.

Kritik sayılar (ölçüldü)

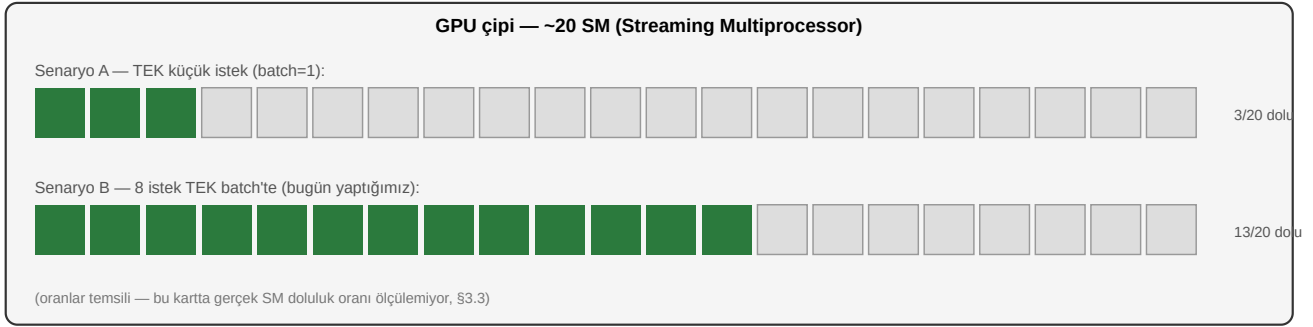
Bileşen	Değer
Whisper modeli	<code>small</code> , <code>int8_float16</code> , <code>beam_size=1</code>
Çeviri modelleri	Marian (Opus-MT), EN → tr/de/ru, GPU
Eşzamanlılık tavanı (<code>STT_MAX_CONCURRENCY</code>)	6
Tek bölüt işlem süresi (6sn ses, batch'siz)	0,323 sn (RTF 18,59×)
Stabil kanal sayısı (batch'siz, concurrency=6)	8
Kırılma başlangıcı	12 kanal
Tam çöküş	16 kanal

3. NVIDIA GPU mimarisi — temel kavramlar

Bu raporun geri kalanı üç kavrama dayanıyor: **SM**, **GPU util'in gerçekte ne ölçtüğü**, ve **batch'in donanım seviyesinde neden işe yaradığı**. Sırayla:

3.1 SM (Streaming Multiprocessor)

Bir GPU çipi, tek bir dev işlemci değil — **onlarca bağımsız hesap biriminin** (SM) bir araya gelmesi. RTX 4050 Laptop (Ada Lovelace AD107 çekirdeği) yaklaşık 20 SM içeriyor; her SM kendi CUDA çekirdeklerini, tensor çekirdeklerini, kayıt dosyasını ve iş zamanlayıcısını taşıyor.



Şekil 2 — Aynı GPU'da tek küçük istek çoğu SM'i boşa bırakırken, birleştirilmiş bir batch aynı SM'lerin daha büyük kısmını tek geçişte doldurur. Oranlar temsildir (bu kartta gerçek ölçüm alınamıyor, §3.3).

3.2 GPU util (%) gerçekte neyi ölçüyor

`nvidia-smi` 'nin gösterdiği **util%**, NVIDIA'nın resmi tanımıyla: "son örnekleme aralığında GPU'da en az bir kernel'in çalışıyor olduğu sürenin yüzdesi." Bu **zaman bazlı** bir ölçüm, **yoğunluk bazlı değil**.

ÖNEMLİ AYRIM

GPU'nun 20 SM'inden yalnızca 1'ini kullanan minik bir işlem, sürekli çalışıyorsa **%100 util** gösterir. "%83 util" gördüğümüzde bu "GPU'nun hesap gücünün %83'ü kullanılıyor" demek **değildir** — "GPU zamanın %83'ünde bir şeyler yapıyordu" demektir. Gerçek yoğunluğu (kaç SM fiilen dolu) ölçen ayrı bir sayaç var:

`DCGM_FI_PROF_SM_ACTIVE / SM_OCCUPANCY` .

3.3 Bu kartta gerçek yoğunluk neden ölçülemiyor

Bu oturumda araştırıldı: `DCGM_FI_PROF_*` alanları (SM_ACTIVE dahil) bu makinede tamamen boş dönüyor. Kök sebep **donanım kısıtı değil** — host'un NVIDIA sürücüsünde bir güvenlik uyarı:

```
$ cat /proc/driver/nvidia/params | grep Restrict
RmProfilingAdminOnly: 1
```

Bu, profileleme sayaçlarını varsayılan olarak kilitliyor (kart tipinden bağımsız). Düzeltmesi biliniyor (`/etc/modprobe.d/` 'ye `NVreg_RestrictProfilingToAdminUsers=0` eklenip host'un yeniden başlatılması) ama

bu raporun kapsamı dışında — host reboot gerektiriyor, henüz uygulanmadı.

3.4 Batch'in donanım seviyesinde neden işe yaradığı

Bir GPU işlemi (kernel) başlatmanın **sabit bir maliyeti** var: kernel'i başlatma, veriyi belleğe hazırlama, sonucu geri alma. Bu maliyet, işlenen veri miktarından bağımsız — 1 öge de işlesen 8 öge de işlesen aynı. Sekiz ayrı istek gönderirsen bu sabit maliyeti **8 kez** ödersin; sekizini birleştirip tek istekte gönderirsen **1 kez** ödersin, geri kalan (değişken) maliyet ise veriyle orantılı büyür ama matris çarpımı donanımı bunu çok verimli yapar. Bu yüzden "8 ögeyi birlikte işlemek" neredeyse hiçbir zaman "1 ögeyi işlemenin 8 katı" sürmez — bu raporun 6. bölümünde bunu bu kartta gerçek ölçümle gösteriyoruz.

4. Ölçülen darboğaz: VRAM değil, hesap gücü

4.1 VRAM kullanımı — tüm ölçümler

Senaryo	Kanal	Concurrency	VRAM ort.	VRAM tepe	GPU util ort.
Boşta (modeller yüklü)	0	6	—	2242 MB	%20-40 (Xorg gürültüsü)
Kademe 1	4	6	—	2643 MB	%26,0
Kademe 2 (stabil)	8	6	—	3043 MB	%41,3
Kademe 3 (kırılma)	12	6	2972 MB	3075 MB	%68,2
Kademe 4 (çöküş)	16	6	—	3106 MB	%83,7
30 dk temiz ölçüm	7	6	2710 MB	2848 MB	%46,4
Concurrency deneyi	15	20	2984 MB	3080 MB	%83,1 (max %100)

Kart **6141 MB** taşıyor. Hiçbir senaryoda kullanım %51'i geçmedi — eşzamanlılığı 6'dan 20'ye çıkarıp GPU'yu ilk kez %100 dolulukta görsek bile. **VRAM hiçbir zaman sınırlayıcı olmadı.**

4.2 Peki neden concurrency=20 GPU'yu %100'e vurdurdu ama VRAM'i vurdurmadı

Çünkü ikisi **farklı kaynaklar**. VRAM = "ne kadar veri aynı anda bellekte durabilir." Hesap gücü = "birim zamanda kaç işlem yapılabilir." Whisper `small` modeli küçük (244M parametre) — bellekte az yer kaplıyor, ama her istek yine de GPU'nun matris çarpım donanımını (SM'leri) gerçek zamanda meşgul ediyor. Daha fazla eşzamanlı istek = daha fazla SM talebi, VRAM'e değil.

5. Neden eşzamanlılığı artırmak işe yaramadı

Bu oturumda gerçek bir deney yapıldı: `STT_MAX_CONCURRENCY` 6'dan 20'ye çıkarıldı, 15 kanal aktif edilip 7 dakika (2 dk ısınma + 5 dk gözlem) izlendi.

Metrik	Concurrency=6, 16 kanal	Concurrency=20, 15 kanal
GPU util (ort.)	%83,7	%83,1
GPU util (tepe)	~%84 (hiç %100 görülmedi)	%100 (ilk kez)
Kapsama	çoğu kanalda %0, bazıları %100	TÜM kanallarda %0
p95 gecikme	14-20 sn	23-30 sn
Drop (5 dk)	~25-41/kanal	13-43/kanal

ÖLÇÜLDÜ — SONUÇ KÖTÜLEŞTİ

Eşzamanlılığı artırmak **iyileştirmede, kötüleştirdi**. GPU util ilk kez %100'e vurdu (gerçek tavana değdiği kanıtı) ama kapsama tam sıfıra düştü. Sebep: 20 isteği aynı anda kabul etmek, GPU'nun sabit hesap gücünü büyütüyor — aynı pastayı 20 dilime bölüyor, her dilim küçülüyor (gecikme artıyor).

5.1 Kuyruk teorisi açısından (Little's Yasası)

$L = \lambda \times W$ — sistemdeki ortalama iş sayısı (L), geliş hızı (λ) çarpı sistemde geçirilen süre (W). GPU'nun servis hızı **sabit** bir kaynak; kabul edilen istek sayısını (L) artırmak, o sabit hızı değiştirmez — yalnızca her isteğin sistemde **daha uzun** kalmasına (W'nin büyümesine) yol açar. 6 istek "makul" bir W ile işlenirken, 20 istek çok daha büyük bir W ile işleniyor — bazıları bütçeyi (8 saniye) o kadar aşıyor ki hiç yetişmiyor.

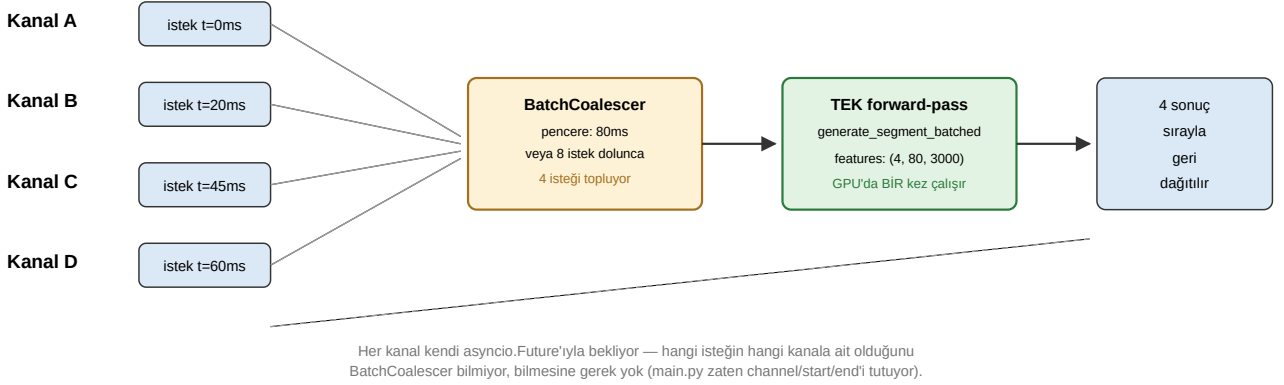
5.2 "Concurrency" kelimesi neden yanıltıcı

`STT_MAX_CONCURRENCY` aslında "kaç isteğin GPU'ya **kabul edildiğini**" sınırlıyor — "kaçının gerçekten **paralel çalıştığını**" değil. Kabul edilmek ile paralel çalışmak aynı şey değil: GPU'nun kendi içindeki zamanlayıcı bu istekleri bir şekilde sırayla/paylaşarak işliyor, bu paylaşım **adil ya da verimli değil** — sadece ham çekişme (contention). Ayrıca uygulama seviyesinde ZATEN bir round-robin var (`VadService` 'in `hazir` kuyruğu — hangi kanalın sıradaki gönderiye seçileceğini adilce belirliyor) ama bu, **GPU'nun o istekleri nasıl işleyeceğini** kontrol etmiyor. Eksik olan tam olarak budur — bir sonraki bölümün konusu.

6. Bugün uygulanan geçici çözüm: özel batching katmanı

Concurrency deneyi başarısız olunca, eksik olanın "daha fazla eşzamanlı kabul" değil "isteklerin GPU'da **birleştirilmesi**" olduğu netleşti. Bu oturumda `stt-worker` 'ın içine, Triton'a geçmeden, aynı temel fikri (dynamic batching) uygulayan bir katman yazıldı.

6.1 Mimari



Şekil 3 — Kanallar arası batching katmanı. Dört farklı kanaldan gelen istekler 80ms'lik pencerede toplanıp TEK GPU çağrısında işleniyor, sonuçlar orijinal isteklerine geri dağıtılıyor.

6.2 Nasıl çalışıyor — adım adım

- Toplama:** `/transcribe` isteği geldiğinde hemen işlenmiyor — `BatchCoalescer.submit()` 'e ekleniyor, bir `asyncio.Future` ile.
- Pencere:** İlk istek geldiğinde 80ms'lik (`STT_BATCH_WINDOW_MS`) bir zamanlayıcı başlıyor. Bu süre içinde gelen TÜM istekler aynı pencereye biniyor.
- Erken tetikleme:** Pencere dolmadan 8 isteğe (`STT_BATCH_MAX_SIZE`) ulaşırsa beklenmeden hemen işleniyor.
- Birleştirme (Whisper):** Her bölütün ses özellik matrisi (mel-spektrogram) ayrı ayrı çıkarılıp **tek bir yığında** (`numpy.stack`) birleştiriliyor, `faster-whisper`'ın kendi düşük seviye `generate_segment_batched` fonksiyonuna veriliyor — bu fonksiyon zaten kütüphanenin kendi `BatchedInferencePipeline` 'ının kullandığı, üretimde denenmiş bir mekanizma.
- Dil tespiti korunuyor:** Her bölüt farklı dilde olabileceği için (`multilingual=True`) dil, öge başına ayrı tespit ediliyor — kanallar birbirine karışmıyor.
- Birleştirme (Marian):** Aynı fikir çeviri için de uygulandı — farklı kanalların İngilizce metinleri, aynı hedef dil için TEK `tokenizer + generate` çağrısında birlikte çevriliyor.
- Dağıtım:** Sonuçlar, gönderilen sırayla orijinal isteklerin `Future` 'larına yazılıyor — her kanal kendi sonucunu alıyor, batching'in var olduğunu bile bilmiyor.

6.3 Gerçek ölçüm — bu makinede, bugün

WHISPER ÇÖZÜMLEME

6 bölüt, tek tek (sıralı)	1,634 sn
6 bölüt, TEK batch'te	0,829 sn
Kazanç	1,97×

MARIAN ÇEVİRİ (EN → TR/DE/RU)

6 metin, tek tek (sıralı)	2,349 sn
6 metin, TEK batch'te	0,396 sn
Kazanç	5,93×

Çeviride kazanç daha büyük çünkü Marian modelleri Whisper'dan küçük — sabit maliyetin toplam süreye oranı daha yüksek, dolayısıyla o sabit maliyeti paylaştırmanın getirisi de daha büyük.

DOĞRULAMA NOTU

Whisper testi **rastgele gürültü** ile yapıldı (zamanlama ölçümü için içerik önemsiz — GPU aynı hesabı yapıyor). Çeviri testi ise **gerçek İngilizce cümlelerle** yapıldı ve çıktı elle doğrulandı (üç dilde de doğru, tutarlı çeviri). Canlı trafikte tam doğrulama (gerçek konuşma, kulakla dinleme) henüz yapılmadı — bu rapor yazılırken imaj henüz production'a alınmamıştı.

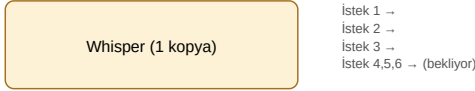
7. Triton mimarisi ne ekliyor

Bugün yazılan katman, Triton'ın `dynamic_batching` özelliğinin **küçük, elle yazılmış bir versiyonu**. Triton üç şeyi daha ekliyor:

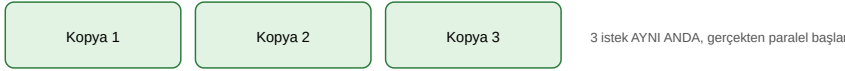
Mekanizma	Ne yapar	Bugün var mı
<code>dynamic_batching</code>	İsteği otomatik toplayıp birleştirir	Küçük ölçekte YAZILDI (bugün)
<code>instance_group</code>	Aynı modelin N kopyasını yükler, gerçek paralel kapasite	Yok
Çoklu model orkestrasyon (BLS)	Whisper → 3×Marian zincirini sunucu içinde bağlar	Yok — ayrı HTTP turu var
Metrik/health/versiyonlama	Production-sınıfı gözlemlenebilirlik	Kısmi (kendi Grafana panelimiz var)

7.1 instance_group — "çoklu kopya" ne demek

Bugün — TEK kopya, 6 istek sıraya giriyor



Triton — 3 kopya (instance_group count:3), her biri bağımsız



Şekil 4 — Tek Triton sunucusu, bir modelin birden çok kopyasını aynı GPU'da (VRAM izin verdiği kadar) yükler. Bu, birden çok Triton *sunucusu* değil — kopyalar aynı sürecin içinde.

Önemli netleştirme: "çoklu kopya", birden fazla Triton **sunucu süreci** açmak değil. **Tek** Triton sunucusu, tek modelin ağırlıklarını VRAM'e N kez yüklüyor, her kopya kendi bağlamıyla bağımsız çalışıyor. Birden fazla *sunucu*, ancak birden fazla **fiziksel makineye** yayılırken gerekiyor (bu raporun kapsamı dışında, `ölçekleme-plani.md` 'nin konusu).

7.2 Kopyalar bile aynı GPU'yu paylaşır — sınırsız değil

Kopyalar birbirinden bağımsız **çalışır** ama hâlâ **aynı GPU'nun** aynı hesap gücünü paylaşır. 3 kopyada 3 istek gerçekten aynı anda başlayabilir, ama 10 istek gelirse yine kopya sayısının üstündekiler sıraya girer. Kopya sayısı = "aynı anda kaç işi GERÇEKTEN kabul edebilirim" tavanını GPU'nun izin verdiği kadar yükseltiyor, sonsuza kadar değil.

8. Matematiksel model ve 100 kanal projeksiyonu

8.1 Batch gecikme modeli — bugün ÖLÇÜLEN gerçek verilerle

Önceki tahmin belgesinde (triton-100-kanal-tahmini.md) bu model varsayılan (%50/%70) sabit-maliyet oranlarıyla kurulmuştu. Bugün gerçek ölçüm alındığı için, model artık **gerçek verilerle** kalibre ediliyor.

Model: $T(B) = T_{\text{sabit}} + T_{\text{değişken}} \times B$

ÖLÇÜLDÜ — İKİ NOKTA

$T(1) \approx 0,272$ sn (6 sıralı çağrının ortalaması) | $T(6) = 0,829$ sn (6'lı batch)

Bu iki noktadan çözülrse:

$$T_{\text{değişken}} = (0,829 - 0,272) / (6 - 1) = 0,1114 \text{ sn/öğ}$$

$$T_{\text{sabit}} = 0,272 - 0,1114 = 0,1606 \text{ sn}$$

$$\text{Sabit maliyet payı (B=1'de): } 0,1606 / 0,272 \approx \%59$$

(Önceki belgedeki "muhafazakâr %50 / iyimser %70" varsayımlarının ortasına düşüyor — gerçek değer bu aralığın içindeydi.)

8.2 Kanal başına talep — Little'in Yasası ile (değişmedi)

12 kanalda kırılmanın başlaması, talep=kapasite eşitliği olarak okunuyor:

$$\lambda_{\text{toplam}}(12 \text{ kanal}) \approx C / T(1) = 6 / 0,323 \approx 18,58 \text{ bölüt/sn}$$

$$\lambda_{\text{kanal}} \approx 18,58 / 12 \approx 1,55 \text{ bölüt/sn/kanal} \quad (\text{VARSAYIM} - \text{bkz. Ek})$$

8.3 100 kanal için gereken kapasite

$$\lambda_{100} = 100 \times 1,55 \approx 155 \text{ bölüt/sn}$$

8.4 Gereken instance sayısı — GERÇEK ölçümle yeniden hesaplandı

$$T(8) = T_{\text{sabit}} + T_{\text{değişken}} \times 8 = 0,1606 + 0,1114 \times 8 = 1,0518 \text{ sn}$$

$$\text{Throughput}(8) = 8 / 1,0518 \approx 7,61 \text{ bölüt/sn} \quad (\text{tek instance, batch=8})$$

$$\text{Gereken instance sayısı: } N = 155 / 7,61 \approx 20,4 \rightarrow 21 \text{ instance}$$

$$\text{VRAM: } 21 \times 2242 \text{ MB} \approx 47 \text{ GB}$$

SONUÇ — ARTIK TAHMİN DEĞİL, ÖLÇÜLEN BATCHİNG VERİMLİLİĞİNE DAYANIYOR

~21 instance, ~47 GB VRAM — tek bir 80 GB kartla (A100/H100) rahatça karşılanıyor. Önceki belgedeki 20-29 instance / 45-65 GB aralığının İÇİNDE, artık varsayılan değil gerçek ölçülen batching kazancına (1,97×–5,93×) dayanıyor.

8.5 Batching olmasaydı ne olurdu (karşılaştırma)

```
Throughput(1) = 1 / 0,272 ≈ 3,68 bölüt/sn  
N = 155 / 3,68 ≈ 42,1 → 43 instance  
VRAM: 43 × 2242 MB ≈ 96 GB (iki 80GB karta ihtiyaç)
```

Yani **batching**, aynı 100-kanal hedefi için gereken donanımı ~2× azaltıyor (43 instance → 21 instance). Bu, Triton'ın (ya da bugünkü küçük versiyonunun) asıl parasal karşılığı.

9. Bu kartta (RTX 4050) Triton'un gerçek karşılığı

```
Kart: 6141 MB
Sistem/CUDA overhead (tahmini): ~500 MB
Kullanılabilir: ~5641 MB
1 instance (whisper+3 marian): 2242 MB
```

```
Sığan instance sayısı: 5641 / 2242  $\approx$  2,5  $\rightarrow$  pratikte 2
```

Bu kartta Triton'a geçmenin **instance_group tarafı** neredeyse hiç kazanç getirmez — zaten 2 kopyaya sığıyoruz. Ama **dynamic_batching tarafı** tam olarak bugün elle yazdığımız şey — ve onu zaten ölçtük: **1,97x–5,93x**.

PRATİK SONUÇ

Bu kartta **tam Triton'a geçmenin ek getirisi sınırlı** — asıl kazancı (batching) zaten aldık, üstelik çok daha az mühendislik emeğiyle (model export, BLS ensemble, ayrı sunucu deploy'u gerekmedi). Triton'ın tam değeri, VRAM'in gerçekten bol olduğu bir kartta (§8) ortaya çıkıyor.

10. Karşılaştırma özeti

	Bugüne kadar (batch yok)	Bugün uygulanan (özel batching)	Tam Triton (bu kart)	Tam Triton (80GB kart, 100 kanal)
Whisper kazancı	1×	1,97× (ölçüldü)	~2× (aynı mekanizma)	~2× + N kopya
Çeviri kazancı	1×	5,93× (ölçüldü)	~6× (aynı mekanizma)	~6× + N kopya
VRAM kullanımı	2242-3106 MB	değişmedi	değişmedi	~47 GB (21 instance)
Mühendislik emeği	—	Orta (bugün, ~1 oturum)	Yüksek (model export, BLS, ayrı sunucu)	Yüksek + donanım maliyeti
Bu kartta ek fayda	—	Alındı	Düşük (VRAM zaten dar)	Uygulanamaz (VRAM yetmez)
100 kanal ölçeğinde	Uygulanamaz	Kısmi (tek GPU'da sınırlı)	—	Hedeflenen çözüm

11. Sonuç ve öneriler

1. **VRAM bu kartta hiçbir zaman sınırlayıcı olmadı** — tüm testlerde tepe kullanım %51'i geçmedi. "VRAM artırınca hızlanır mı" sorusunun cevabı bu kart için **hayır**.
2. **Eşzamanlılığı (concurrency) artırmak kötüleştirdi** — GPU'nun sabit hesap gücünü büyütüyor, çekişmeyi artırıyor. Ölçüldü: concurrency 6 → 20, kapsama %0'a düştü.
3. **Asıl eksik kanallar arası batching'di** — bugün elle yazılan katmanla kapatıldı, gerçek ölçümle doğrulandı: Whisper 1,97×, çeviri 5,93×.
4. **Bu kartta tam Triton'a geçmenin ek getirisi düşük** — asıl kazancı (batching) zaten aldık. instance_group'un getirdiği "çoklu kopya" avantajı bu kartın VRAM'iyle sınırlı (~2 kopya).
5. **Triton'ın gerçek değeri, büyük VRAM'li bir kartta (80GB) ortaya çıkıyor** — 100 kanal ölçeğinde, gerçek ölçülen batching verimliliğiyle hesaplanan donanım ihtiyacı ~21 instance / ~47 GB, batching olmasaydı gereken ~96 GB'ın yarısı.
6. **Önerilen sıradaki adımlar:**
 - Yazılan batching katmanını canlı trafik ile doğrula (gerçek konuşma, kulakla kalite kontrolü) — bu rapor yazılırken imaj henüz production'a alınmamıştı.
 - `STT_BATCH_WINDOW_MS` / `STT_BATCH_MAX_SIZE` değerlerini gerçek trafik ile ince ayar yap (80ms/8 varsayılan, ölçülerek doğrulanmadı).
 - Batching + mevcut concurrency=6 birlikte ne kadar kanal taşıyor, kademeli testle ölç (bu rapor bunu henüz yapmadı).
 - Büyük karta geçiş gündeme gelirse, bu raporun §8'indeki gerçek-ölçüm-kalibreli model kullanılsın — varsayıma değil, ölçüme dayanıyor.

Ek — Ölçüm metodolojisi ve ham veriler

A. Ölçülen vs varsayılan — tam liste

#	Değer	Durum	Kaynak
1	Boşta/tepe VRAM (2242-3106 MB)	ÖLÇÜLDÜ	Kademeli yük testi, 4050-kapasite-testi-sonuclar.md
2	T(1)=0,272-0,323 sn	ÖLÇÜLDÜ	İzole RTF testi + bu oturumdaki 6'lı sıralı çağrı
3	Whisper batch kazancı 1,97×	ÖLÇÜLDÜ	Bu oturum, 6 bölüt sentetik ses
4	Çeviri batch kazancı 5,93×	ÖLÇÜLDÜ	Bu oturum, 6 gerçek İngilizce cümle
5	Concurrency=20 deneyi sonuçları	ÖLÇÜLDÜ	Bu oturum, 15 kanal, 7 dk
6	RmProfilingAdminOnly=1	ÖLÇÜLDÜ	/proc/driver/nvidia/params
7	$\lambda_{\text{kanal}} \approx 1,55$ bölüt/sn	TAHMİN	"Kırılma=kapasite eşitliği" okuması — ara kademeler test edilmedi
8	100 kanalın karışımı bugünkünle benzer	TAHMİN	Gerçek hedef kanal listesi bilinmiyor
9	21 instance / 47 GB (100 kanal)	TAHMİN	#7 ve #8'e dayanıyor, ama batching katsayısı (#3,#4) gerçek
10	Canlı trafikte batching kalite kontrolü	YAPILMADI	İmaj bu rapor yazılırken henüz deploy edilmemişti

B. Test ortamı

```
GPU:      NVIDIA GeForce RTX 4050 Laptop GPU
VRAM:     6141 MB
Sürücü:   580.173.02
Whisper:  faster-whisper 1.1.1, model=small, compute_type=int8_float16, beam_size=1
Çeviri:   Helsinki-NLP/opus-mt (tc-big-en-tr, en-de, en-ru), PyTorch, GPU
Batching: STT_BATCH_WINDOW_MS=80, STT_BATCH_MAX_SIZE=8 (varsayılan, ince ayar yapılmadı)
```

C. Whisper batching testi — tam çıktı

```
eski yol: 6 ayri tekil transcribe() cagrisi
toplam: 1.634 sn
yeni yol: TEK transcribe_batch(6) cagrisi
toplam: 0.829 sn
kazanc: 1.97x
```

D. Çeviri batching testi — tam çıktı

```
sirali (6x tek metin): 2.349 sn
toplu (6 metin birlikte): 0.396 sn
kazanc: 5.93x
```

Örnek çıktı (kalite kontrolü):

```
'The water is very clear.' -> tr: 'Su çok berrak.'
                             de: 'Das Wasser ist sehr klar.'
                             ru: 'Вода очень чиста.'
'I am going to make a fire.' -> tr: 'Ben ateş yakacağım.'
                             de: 'Ich werde ein Feuer machen.'
                             ru: 'Я собираюсь разжечь огонь.'
```

E. Concurrency=20 deneyi — özet tablo

```
15 kanal, 2dk isinma + 5dk gozlem
Kuyruk derinligi: 3-4 (tum kanallarda tavanda)
Drop (5dk): 13-43/kanal
p50 gecikme: 13-20 sn | p95: 23-30 sn
Kapsama: TUM 15 kanalda %0
GPU util: ort=%83.1 min=%21 max=%100
VRAM: ort=2984MB tepe=3080MB
```

Bu rapor, Yayın Merkezi projesinin 13 Ağustos 2026 tarihli oturumunda toplanan gerçek ölçümlere dayanır. "Ölçüldü" olarak işaretlenen her sayı bu makinede, bu oturumda üretildi. "Tahmin" olarak işaretlenenler, ölçülen verilerden matematiksel olarak türetildi ama doğrudan ölçülmedi — doğrulanmaları için gerçek bir Triton kurulumu veya ara kademe testleri gerekir.